



Módulo 2 — Value Objects (Parte 2 de 3)

Curso: Arquitectura de Software Profesional con Clean Architecture, DDD y SOLID

*Caso de uso guía: **CU-CLI-001 — Alta de Cliente***

Temas de esta parte

Tema 5	Método Create() en Value Objects
Tema 6	Excepciones en Value Objects
Tema 7	Record base ValueObject

Tema 5 — Método Create() en Value Objects

1. ¿Qué es?

El método `Create()` es un método estático de fábrica que reemplaza al constructor público en los Value Objects. Es el único punto de entrada para crear una instancia válida del record. Tiene tres responsabilidades en orden estricto:

1. **Recibir** el valor crudo del exterior (puede ser nulo o inválido).
2. **Validar** que el valor cumpla las reglas del negocio.
3. **Crear y devolver** la instancia del record solo si pasó la validación.

2. Analogía

Un **inspector de aduana** es la única puerta de entrada al país. No importa quién intente ingresar: todos pasan por el inspector. Si la documentación no es válida, se rechaza en el momento. Si es válida, se emite el permiso de ingreso. El método `Create()` es ese inspector: nadie entra sin pasar por él.

3. ¿Por qué no usar el constructor directamente?

Constructor público	Método Create() estático
Permite crear instancias sin validación	Toda instancia pasa obligatoriamente por la validación
No puede tener nombre descriptivo	<code>Email.Create()</code> comunica intención claramente
No puede normalizar el valor antes de guardar	Puede aplicar <code>Trim()</code> , <code>ToLower()</code> , etc. antes de construir
Riesgo de instancias en estado inválido	Imposible: o devuelve una instancia válida o lanza excepción

4. Estructura del método Create() — 4 pasos

```
1 public sealed record Email : ValueObject
2 {
3     // — Paso 1: propiedad de solo lectura —————
4     public string Value { get; }
5
6     // — Paso 2: constructor privado — nadie puede instanciar desde afuera —
7     private Email(string value) => Value = value;
8 }
```

```
9 // — Paso 3: validaciones — lanza excepción si el valor no es válido —
10 public static Email Create(string? value)
11 {
12     if (string.IsNullOrWhiteSpace(value))
13         throw new ArgumentException("El email es obligatorio.", nameof(value));
14     if (!value.Contains("@"))
15         throw new ArgumentException("El email debe contener @.", nameof(value));
16
17     // — Paso 4: normalizar y construir — solo si las validaciones pasaron —
18     return new Email(value.Trim().ToLower());
19 }
20 }
```

5. Flujo de ejecución

```
Email.Create(" Juan@Mail.COM ")
→ IsNullOrWhiteSpace? No → continúa
→ Contains("@")? Sí → continúa
→ Trim() + ToLower() → "juan@mail.com"
→ new Email("juan@mail.com")
→ instancia válida devuelta

Email.Create("")
→ IsNullOrWhiteSpace? Sí
→ ArgumentException: "El email es obligatorio."
```

6. Normalización antes de construir

El método `Create()` puede transformar el valor antes de almacenarlo. Esto garantiza que el record siempre contiene el dato en su forma canónica, independientemente de cómo llegó el input del usuario.

```
1 // Normalizaciones típicas en el Create():
2 value.Trim()           // elimina espacios al inicio y al final
3 value.ToLower()        // normaliza a minúsculas (email)
4 value.ToUpper()        // normaliza a mayúsculas (documento, código)
5 value.Trim().ToUpper() // combinación: limpiar + estandarizar
```

Resumen del Tema 5

- `Create()` es la única puerta de entrada: constructor privado + fábrica estática.
- Garantiza que toda instancia del record sea válida — no existe instancia inválida.
- Recibe el valor crudo, valida, normaliza y construye en ese orden.



- Si la validación falla, lanza una excepción antes de construir nada.

Tema 6 — Excepciones en Value Objects

1. ¿Por qué excepciones y no códigos de error?

Cuando un Value Object recibe un valor inválido, el contrato del dominio ha sido violado. No es un caso de negocio esperado: es un error de programación o de integración. La respuesta correcta es **lanzar una excepción** que interrumpa la operación inmediatamente, no devolver un código de error que podría ignorarse silenciosamente.

2. Analogía

Un **contrato firmado con cláusulas ilegales** es nulo de pleno derecho: no puede existir. Nadie lo acepta parcialmente ni lo "devuelve con un código -1". Se rechaza en el momento de la firma. El record funciona igual: si los datos no son válidos, el objeto no nace.

3. Excepciones estándar de .NET vs excepciones de dominio

Tipo	Cuándo usarla	Ejemplo
<code>ArgumentException</code>	El argumento recibido no cumple el contrato del método	Email vacío, formato inválido
<code>ArgumentNullException</code>	El argumento es nulo y no se acepta nulo	<code>value</code> es <code>null</code>
Excepción de dominio propia	La violación de regla de negocio merece un tipo semántico específico	<code>InvalidEmailException</code> , <code>UnderageCustomerException</code>

En este módulo usamos `ArgumentException` estándar de .NET. En el **Módulo 3** se reemplazarán por excepciones de dominio propias que expresan con mayor precisión la violación de cada regla de negocio.

4. Patrón de validación en el Create()

```
1 // — Patrón: una validación → una excepción → mensaje específico —
2 public static BirthDate Create(DateOnly value)
3 {
4     // Regla 1: la fecha no puede ser en el futuro
5     if (value > DateOnly.FromDateTime(DateTime.Today))
```

```
6         throw new ArgumentException("La fecha de nacimiento no puede ser futura.",
7         nameof(value));
8
9         // Regla 2: el cliente debe ser mayor de edad
10        var age = DateTime.Today.Year - value.Year;
11        if (age < 18)
12            throw new ArgumentException("El cliente debe ser mayor de edad.", nameof(value));
13
14        return new BirthDate(value);
15    }
```

5. Propagación de la excepción

La excepción lanzada en el `Create()` sube automáticamente por la pila de llamadas hasta el primer manejador que la capture. Ningún código intermedio necesita saber que existió la validación.

```
BirthDate.Create(fechaInválida)
→ lanza ArgumentException("El cliente debe ser mayor de edad.")
→ Customer.Create() no la captura → sube
→ RegisterCustomerUseCase no la captura → sube
→ CustomerController la captura → devuelve 400 Bad Request
```

6. Ventajas de este enfoque

- **Fail fast:** el error se detecta en el momento exacto en que se intenta crear el dato inválido.
- **Sin estado inválido:** si `Create()` retorna, el record es válido — sin excepciones.
- **Mensajes semánticos:** el mensaje de la excepción describe exactamente qué regla se violó.
- **Sin código repetido:** la validación existe en un solo lugar; el mensaje también.

Resumen del Tema 6

- Los Value Objects usan excepciones, no códigos de error ni valores nulos de retorno.
- Cada regla violada lanza su propia excepción con un mensaje específico.
- En este módulo: `ArgumentException`. En Módulo 3: excepciones de dominio propias.



- La excepción propaga sola; el controlador la transforma en la respuesta HTTP apropiada.

Tema 7 — Record base ValueObject

1. ¿Qué es y para qué sirve?

ValueObject es un **record abstracto marcador**: una declaración de una sola línea que todos los Value Objects del dominio heredan. No aporta lógica ni métodos: su único propósito es **expresar en el modelo que un tipo es un Value Object**.

```

1 // — CommerX.Domain/Common/ValueObjects/ValueObject.cs —————
2 namespace CommerX.Domain.Common.ValueObjects;
3
4 /// <summary>
5 /// Marcador abstracto para todos los Value Objects del dominio.
6 /// La igualdad estructural la provee el compilador mediante record.
7 /// </summary>
8 public abstract record ValueObject;
```

2. ¿Por qué es un record abstracto y no simplemente nada?

Técnicamente, cada **sealed record** funciona sin herencia: la igualdad estructural y la inmutabilidad están en todos los records por diseño del lenguaje. Sin embargo, **el record base aporta valor arquitectónico** que va más allá de lo técnico. Existen cinco razones concretas:

1. Semántica en el modelo	sealed record Email : ValueObject comunica inmediatamente que Email es un Value Object del dominio. Sin la base, esa intención queda implícita solo en la documentación.
2. Seguridad de tipos	Permite escribir métodos y restricciones genéricas que acepten exclusivamente Value Objects: where T : ValueObject . Sin la base, no hay forma de distinguir un VO de cualquier otro record.
3. Polimorfismo controlado	Hace posible operar sobre VOs sin conocer su tipo concreto. Por ejemplo: un validador genérico, un mapper o un logger pueden recibir ValueObject y actuar sobre cualquier VO.
4. Punto de extensión DDD	Si en el futuro se necesita agregar comportamiento transversal a todos los VOs (por ejemplo, serialización, auditoría o conversión), existe un único lugar donde hacerlo sin modificar cada record individualmente. Esto respeta OCP.
5. Convención de equipo	Establece una regla de arquitectura verificable: todo Value Object del dominio hereda de ValueObject . Cualquier revisión de código o herramienta de análisis puede detectar VOs que no cumplen la convención.

3. ¿Por qué abstracto?

La palabra clave `abstract` impide que `ValueObject` sea instanciado directamente. Solo puede existir a través de sus tipos concretos: `Email`, `Document`, `BirthDate`, etc. Esto refuerza el contrato: un *Value Object genérico* sin contenido no tiene sentido en el dominio.

```
1 // abstract impide instanciar la base directamente:
2 new ValueObject(); // no compila - abstract no se puede instanciar
3
4 // Solo los records concretos pueden existir:
5 var email = Email.Create("juan@mail.com"); // correcto
6
7 // El tipo base permite polimorfismo seguro:
8 ValueObject vo = Email.Create("juan@mail.com"); // válido
9 vo is Email; // true - pattern matching funciona sobre la jerarquía
```

4. ¿Por qué record y no otra cosa?

El marcador debe ser del mismo tipo que sus derivados para que la herencia funcione correctamente. En C# 9+, un `record` solo puede heredar de otro `record`. Por lo tanto, si los Value Objects son `sealed record`, la base **también debe ser un `record`**.

```
1 // Regla del lenguaje: un record solo puede heredar de otro record
2 public sealed record Email : ValueObject // correcto - ValueObject es record
3 public sealed record Email : SomeClass // no compila - un record no hereda de una clase
```

5. Value Object completo heredando del marcador

```
1 // — CommerX.Domain/Customers/ValueObjects/Email.cs —————
2 namespace CommerX.Domain.Customers.ValueObjects;
3
4 public sealed record Email : ValueObject
5 {
6     public string Value { get; }
7     private Email(string value) => Value = value;
8
9     public static Email Create(string? value)
10    {
11        if (string.IsNullOrEmpty(value))
12            throw new ArgumentException("El email es obligatorio.", nameof(value));
13        if (!value.Contains("@"))
14            throw new ArgumentException("El email debe contener @.", nameof(value));
15        return new Email(value.Trim().ToLower());
16    }
17 }
```

6. Lo que el compilador genera automáticamente

Gracias a que `Email` es un `record`, el compilador genera sin ningún código extra:

```

1  var e1 = Email.Create("juan@mail.com");
2  var e2 = Email.Create("juan@mail.com");
3
4  e1 == e2           // true - operator == generado por el compilador
5  e1.Equals(e2)      // true - Equals() generado por el compilador
6  e1.GetHashCode()   // hash basado en Value - generado por el compilador
7  e1.ToString()      // "Email { Value = juan@mail.com }" - para logs/debug
8
9  var set = new HashSet<Email> { e1, e2 };
10 set.Count; // 1 - e1 y e2 tienen el mismo hash y son iguales

```

7. Estructura final de archivos

```

CommerX.Domain
+---Common
|   +---Entities
|   |   └─ BaseEntity.cs      ← abstract class (base para entidades)
|   +---ValueObjects
|   |   └─ ValueObject.cs     ← abstract record (marcador para Value Objects)
+---Customers
+---Entities
|   └─ Customer.cs           ← sealed class : BaseEntity
+---ValueObjects
|   └─ Email.cs              ← sealed record : ValueObject
|   └─ FullName.cs          ← sealed record : ValueObject
|   └─ ...                  ← cada VO hereda del marcador

```

Resumen del Tema 7

- `public abstract record ValueObject;` — una línea que aporta semántica, seguridad de tipos y extensibilidad.
- `abstract` impide instanciarla directamente: solo existen los records concretos.
- La igualdad estructural la provee el compilador en cada `record` — no la base.
- Un `record` solo puede heredar de otro `record` — por eso la base es un record y no otra cosa.
- La base es el punto de extensión para comportamiento transversal futuro (serialización, logging, etc.).



Continúa en **Módulo 2C** — Implementación de los 7 Value Objects, actualización de **Customer** y actividades prácticas.